

Atmospheric Scattering

For this assignment you'll be simulating a regular everyday phenomenon, you will do so by learning about how computers draw pixel by pixel. By the end you will have built a physically based atmospheric rendering paradigm from the ground up with basic atmospheric physics and progressively upgrade the architecture to handle stochastic probability, multiple scattering.

Chapter 1: Rayleigh Scattering & Transmittance

Light as it moves in atmosphere, can collide with air molecules, these collisions are essential for the phenomenon of scattering. There are two things happening

- **Transmittance:** The light slowly loses energy the further it travels.
- **Rayleigh Scattering:** The light scatters relatively uniformly, but it heavily favors shorter blue wavelengths.

The probability of a photon scattering toward the camera is governed by the Rayleigh Phase Function:

$$P_{Rayleigh}(\theta) = \frac{3}{16\pi}(1 + \cos^2 \theta) \quad (1)$$

The loss of light over a distance s is defined by the Beer-Lambert Law (Transmittance):

$$T = \exp\left(-\int_0^s \beta(h) ds'\right) \quad (2)$$

Where $\beta(h)$ is the density of the air, which drops exponentially the higher you go (h).

Your Task:

Use any OOP of your choice along with a graphics API, we recommend Python+OpenGL for beginners. Link up your language with the graphics API and set up a scene. Using the above two equations, write a function that calculates the Optical Depth by numerically marching along a vector and summing the exponential density at each step.

Chapter 2: Single Scattering

Having setup how the light degrades, we move on to simulate the light hitting the camera. In this stage, we assume a photon only bounces *once* (hitting a gas molecule and immediately bouncing into camera).

To calculate the color of light as it passes through a participating medium for a single pixel, we use the Volumetric Rendering Equation:

$$L_{in} = \int_0^s T(s') \cdot \beta(s') \cdot P(\theta) \cdot L_{sun} ds' \quad (3)$$

Your Task:

Set up a deterministic loop in your shader. For a single pixel, march along the camera's view ray. At each step, calculate a direct shadow ray to the sun, find the transmittance, multiply it by the Rayleigh phase function as you would have implemented before, and accumulate the resulting light. You should see a basic, glowing atmosphere!

Chapter 3: Mie Scattering

But we don't just have gasses in atmosphere; there are also aerosols (dust, pollution, water vapor). Unlike gas particles, these are bigger in size, They scatter all color equally creating a white haze, This phenomenon is **Mie Scattering**. Unlike Rayleigh these particles are **anisotropic** ie. they act like microscopic shotguns, violently blasting light forward rather than uniformly.

This forward directionality is modeled using the Henyey-Greenstein phase function, driven by the asymmetry parameter g (where $g = 0.76$ is standard forward-scattering dust):

$$P_{Mie}(\theta) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g \cos \theta)^{3/2}} \quad (4)$$

Your Task:

Add a separate density calculation for Mie particles, celebrate that they sit much lower to the ground than gases. Add the Henyey-Greenstein phase function to your deterministic loop. You should obtain a haze around your source of light!

Chapter 4: Monte Carlo & Inverse Transform Sampling

We've been using Riemann Sums until now, which have worked great assuming light collides only once. But that's not what happens in real life, light in reality keeps bouncing. Using a deterministic loop to simulate these bounces, for N steps you will calculate N^{Bounces} , a 1080p screen has 2M+ pixels, can you see how this becomes a problem even for an dedicated GPU.

To actually simulate light we switch to the clever technique of **Monte Carlo Integration**. However, if we pick pure random directions for Mie scattering, the variance will destroy the image with noise, We must bias our randomness using **Importance Sampling**.

We use Inverse Transform Sampling. By calculating the Cumulative Distribution Function (CDF) of the Henyey-Greenstein equation and inverting it, ξ means many things in math, but here it is simply a uniform random number from 0.0 to 1.0.

$$\cos \theta = \frac{1}{2g} \left(1 + g^2 - \left(\frac{1 - g^2}{1 - g + 2g\xi} \right)^2 \right) \quad (5)$$

Because our probability generation now perfectly matches physical reality, the phase function completely cancels out of the Monte Carlo estimator:

$$\langle L_{scatter} \rangle = \frac{1}{N} \sum_{k=1}^N \frac{L(\omega_k)P(\omega_k)}{P(\omega_k)} = \frac{1}{N} \sum_{k=1}^N L(\omega_k) \quad (6)$$

However if you do just this, you will observe that your screen is a mess of flashing static, to deal with this we need to setup an **Accumulation Buffer**, We do so by saving the state of the previous frame, now every new frame we calculate one noisy pixel, retrieve the historical average from the buffer, blend them mathematically causing in a new smoother average to the buffer.

Let C_N be the accumulated color at frame N, and c_{new} be the noisy color generated this exact frame.

$$C_N = C_{N-1} \cdot \left(1 - \frac{1}{N} \right) + c_{new} \cdot \left(\frac{1}{N} \right) \quad (7)$$

Your Task:

Implement a recursive Monte Carlo **for** loop that allows a photon to randomly bounce up to N(4) times. You will see that your image has become noisy, do not fret as this is the nature of Monte Carlo Integration, once you implement Importance Sampling you will observe that the noise converges to a smooth frame as the frames accumulate.

Recommended Resources

- **Quick Atmospheric Scattering Demonstration**
- **Breakdown of the numerical integration**
- **Monte Carlo Integration Explained**
- **Generating Samples from a Custom PDF**
- **A great visualisation of inverse transform sampling**